CS-3813 Project Final Report

25% total (5 pages):

1. Title & group members
2. Abstract (150 words)
3. Introduction/ motivation
4. Objectives
5. Design/ development/ testing
6. Results/ discussion
7. Conclusion
8. References
9. Member responsibilities
10. Time (no. of hours)

1. Title & group members:

Title: MIPS vs RISC Energy Efficiency

Group: Darin, Fady & Tonye.

2. Abstract (150 words):

Energy efficiency is an extremely important aspect in computer design. This is especially true for devices, small computers and powerful computing systems. We compared the energy efficiency of the MIPS computer design with RISC designs. We focused on math operations: addition, subtraction, multiplication, division and modulo. MIPS was one of the RISC designs initially designed. It uses a model to load and store data, has 32-bit instructions and executes instructions in a straightforward pipeline. These features help reduce energy use per instruction. However, MIPS needs instructions for complex tasks. It also lacks hardware for multiplication and division operations. Furthermore, it does not support power-saving techniques like dynamic voltage and frequency scaling or low-power idle states. Modern RISC designs like RISC-V and ARM improve on the RISC ideas. They add power management, better compiler optimizations and hardware acceleration for complex tasks. We analysed how many instructions are needed how many cycles per instruction and power management features for each math operation. Our results show that modern RISC designs are more energy efficient especially, for complex tasks. These findings help in choosing the computer design for situations where energy is limited.

3. Introduction/ motivation:

MIPS and RISC architectures are two of the most important ideas in the history of processors. They both start with the basic idea: making the instructions simpler so the

processor can work faster and use less hardware. Over time these two ideas have developed in ways especially when it comes to using energy efficiently particularly for tasks that need a lot of math.

MIPS was one of the successful RISC processors and it was developed at Stanford University in the early 1980s. The people at MIPS Computer Systems sold it to others. The design of MIPS is based on a set of instructions that always stay the same and a simple five-step process: getting the instructions figuring out what they mean doing the work getting things from memory and writing back the answers. Other RISC architectures, like ARM and RISC-V have changed a lot from the idea. They have features and ways to manage power that MIPS does not have.

Now that we have many devices that run on batteries and data centres are using more power and people are worried, about how much energy we use it is just as important to think about how much energy a processor uses as how fast it is. The basic math operations. Like adding, subtracting, multiplying, dividing and finding the remainder. Are what every computation is based on. If we can understand how well each architecture handles these math operations, we can get an idea of how much energy the whole system will use.

4. Objectives:

The main goals of this project are:

- To see how instructions it takes to do each of the five arithmetic operations. Addition, subtraction, multiplication, division and modulo. In MIPS assembly and in a modern RISC implementation.

- To find out how energy each operation uses using things like CPI values and clock frequency to make an estimate.

- To figure out how energy use changes when we do more arithmetic instructions, like in a loop and to find the point where the differences between the two architectures really start to matter.

- To look at how things like power management features. DVFS, hardware multiply and divide units and low power idle states. Affect the total energy cost of doing lots of arithmetic operations in modern RISC versus MIPS.

- To make a program that tests and compares how instructions it takes to do each operation in MIPS assembly and, in a modern RISC style so we can see which one more energy is efficient.

5. Design/ development/ testing:

The table below shows the MIPS instructions and the equivalent RISC-V instructions for five operations along with their typical cycle counts.

The table below shows the MIPS instructions and the equivalent RISC-V instructions for five operations along, with their typical cycle counts.

Operation	MIPS Instruction	MIPS Cycles	RISC-V Instruction	RISC-V Cycles	Notes
Addition	ADD $t0, $t1, $t2	1	ADD x5, x6, x	1	Equal cost; both complete in 1 cycle
Subtraction	SUB $t0, $t1, $t2	1	SUB x5, x6, x	1	Equal cost; both complete in 1 cycle
Multiplication	MULT $t1, $t2 (→ HI/LO) MFLO $t0	16 cycles	MUL x5, x6, x	4 cycles	MIPS needs 2 instrs + pipeline stall; RISC-V uses 1 instr, hardware M-ext
Division	DIV $t1, $t2 (→ HI/LO) MFLO $t0 (quotient)	35 cycles	DIV x5, x6, x	12 cycles	MIPS stalls entire pipeline; RISC-V hardware divider, fewer stall cycles
Modulo	DIV $t1, $t2 MFHI	36 cycles	REM x5, x6, x	12 cycles	MIPS requires extra

	$t0 (remainder)				MFHI; RISC-V REM is a dedicated 1-instruction operation

```
CS-3813 RISC-V Arithmetic Benchmark
-----------------------------
Iterations per operation: 100
-----------------------------
[ADD]  47 + 6 = 53
[SUB]  47 - 6 = 41
[MUL]  47 * 6 = 282
[DIV]  47 / 6 = 7
[MOD]  47 % 6 = 5
-----------------------------

Instruction analysis (1 iteration):
  ADD  instructions: 1 | cycles: 1
  SUB  instructions: 1 | cycles: 1
  MUL  instructions: 1 | cycles: ~4 avg (M-ext, no stall)
  DIV  instructions: 1 | cycles: ~12 avg (M-ext hardware divider)
  REM  instructions: 1 | cycles: ~12 avg (dedicated REM instruction)

-- program is finished running (0) --
```

Console

```
CS-3813 MIPS Arithmetic Benchmark
-----------------------------
Iterations per operation: 100
-----------------------------
[ADD]  47 + 6 = 53
[SUB]  47 - 6 = 41
[MUL]  47 * 6 = 282
[DIV]  47 / 6 = 7
[MOD]  47 % 6 = 5
-----------------------------

Instruction analysis (1 iteration):
  ADD  instructions: 1  | cycles: 1
  SUB  instructions: 1  | cycles: 1
  MUL  instructions: 2  | cycles: ~16 avg (HI/LO stall)
  DIV  instructions: 2  | cycles: ~35 avg (pipeline stall)
  MOD  instructions: 2  | cycles: ~36 avg (DIV + MFHI)
```

To estimate relative energy consumption, we use the standard dynamic power formula:

$$E = P \times T = (C \times V^2 \times f) \times (N_instructions \times CPI) / f = C \times V^2 \times N_instructions \times CPI$$

Where E is the energy used for each operation C is the load, which depends on the circuit area that switches per operation V is the voltage supplied f is the clock frequency N_instructions is how many instructions are needed for the operation and CPI is the average number of cycles per instruction.

For a comparison we set C and V to 1.0 for the MIPS baseline and express energy as a relative value. The main factors that differentiate operations are N_instructions. Cpi. We also consider the advantage of RISC, in reducing voltage and power consumption. When the voltage is reduced by 15% the power drops by 28% because of the V^2 effect. This is discussed in the Power Wall slides.

6. Results/ discussion:

6.1 Addition and Subtraction

When we do addition and subtraction the energy it takes is basically the same for MIPS and modern RISC architectures. They both do these operations in one step. They do not need to get anything from memory. This means that the computer does not have to wait for anything long as it has the numbers it needs. The way MIPS and RISC-V do addition and subtraction is very similar. They are so similar that it does not matter which one we use because they use the amount of

energy. This makes sense because addition and subtraction are the things that a computer can do. The people who made RISC architectures wanted to make sure that simple things are fast, and they did a job, with both MIPS and modern RISC architectures. Addition and subtraction are operations and both MIPS and RISC-V architectures can do them quickly.

6.2 Multiplication

When we do multiplication, we see a difference between MIPS and modern RISC. In MIPS the multiplication instruction does not put the answer directly into a register that we can use. Instead, the answer is put into two registers called HI and LO. The HI register gets the 32 bits, and the LO register gets the low 32 bits. Then we have to use another instruction called MFLO to move the 32 bits into a register that we can use. This means we have to use two instructions, and the computer has to wait for the multiplication to finish which can take a time usually between 2 and 32 cycles. This makes multiplication use a lot of energy than it would in a modern RISC computer.

RISC-V is a type of RISC computer that has an instruction called MUL. The MUL instruction does the multiplication. Puts the answer directly into a register and it only takes between 1 and 8 cycles. The programmer does not have to worry about the computer waiting for the multiplication to finish. When we compare the energy used by MIPS and RISC-V for multiplication we see that RISC-V uses a lot of energy. The ratio of energy used by MIPS to energy used by RISC-V is 8 to 1. This is because MIPS has to wait for a time for the

multiplication to finish which uses a lot of energy. Multiplication in RISC-V is more efficient than multiplication, in MIPS.

6.3 Division and Modulo

The Division and Modulo operations have a big problem with energy. When the processor does Division, in MIPS it has to stop everything for 32 to 38 cycles. This means the processor cannot do any work while it is doing the Division.

The result of the Division and the remainder are put in two places called LO and HI. The person writing the program has to use extra instructions to get these results. They have to use MFLO to get the result of the Division or MFHI to get the remainder.

So, to do Modulo in MIPS the processor has to do two things: it must do the Division, which takes a time and then it must do MFHI. This takes a total of two instructions. The processor must wait a long time. Division and Modulo operations are very slow.

7. Conclusion:

This study looked at how the MIPS architecture and RISC-V use energy when they do basic math problems like addition, subtraction, multiplication, division and modulo. The results show that the energy used depends on the type of math problem.

For addition and subtraction, the MIPS architecture and RISC-V are much the same when it comes to energy use. Both of them can do these math problems in one step without any delays so one is not better than the other in this case.

When it comes to multiplication the MIPS architecture uses a lot energy than RISC-V. This is because the MIPS architecture has to use registers and it takes several steps to do the math problem, which wastes energy.

For division and modulo the MIPS architecture also uses a lot energy than RISC-V. This is because it takes a time to do the division and the MIPS architecture needs special instructions to get the results.

When you do a lot of these math problems the difference in energy use really adds up. If you are doing math problems that involve multiplication, division or modulo the MIPS architecture will use a lot of energy than a newer RISC-V system. If you also consider other features like adjusting the speed of the system and turning off parts of it to save energy the RISC-V system looks even better.

The conclusion is clear: while the MIPS architecture was important in the past and is still useful for teaching newer RISC-V designs are an improvement when it comes to using energy efficiently. This is especially true for the types of math problems that are used a lot in real-world applications, like multiplication, division and modulo. For things, like devices, sensors and other small systems that run on batteries using a newer RISC-V

system is a better choice because it uses less energy and works better.

8. References:

Computer Organization & Design: The Hardware/Software interface by John L. Hennessy & David A. Patterson, Morgan Kaufmann Publishers, RISC-V Edition, Year 2018.

Computer Organization & Design: The Hardware/Software interface by John L. Hennessy & David A. Patterson, Morgan Kaufmann Publishers, Fifth Edition, Year 2014

Suárez, D., Almeida, F. & Blanco, V. Comprehensive analysis of energy efficiency and performance of ARM and RISC-V SoCs. *J Supercomput* **80**, 12771–12789 (2024). https://doi.org/10.1007/s11227-024-05946-9

Anita K. Jones, Thomas R. Gross, John L. Hennessy, Steven A. Przybylski, Christopher Rowen, (1988), Measurement and evaluation of the MIPS architecture and processor https://dl.acm.org/doi/abs/10.1145/45059.45060

9. Member responsibilities:

Member	Responsibilities
Darin	Literature review on MIPS architecture; instruction count analysis for ADD, SUB; energy model derivation; editing and formatting of final report

Tonye	Research on modern RISC-V architecture features; DVFS and power management analysis; instruction comparison table; Results and Discussion section
Fady	Abstract and Introduction drafting; Objectives; Conclusion; assembly code development for benchmarking program; presentation preparation

10. Timeline (no. of hours per deliverable):

Month	Hours of work	Major Task
1. January	2 – 5 hrs	Brainstorming topic
2. February	4 – 7 hrs	Project Interim report/ proposal
3. March	8 hrs	Final draft of the study/ paper
4. April	3 hrs	Creating of in-class Presentation

*(The time mentioned here is an average of how many how hours the team has spent to accomplish each of the tasks assigned.)